26CSE0106 - Introduction to Web Technologies

Flex, RWD, Animation

Department of Computer Science and Engineering
Chitkara University, Punjab

CHITKARA UNIVERSITY

Flex

Department of Computer Science and Engineering
Chitkara University, Punjab

Contents

CHITKARA UNIVERSITY

- Background
- Basics and terminology
- Absolute and Relative Flex
- Auto-margin alignment
- Switching Flex Direction
- Flexbox Solved That
- Building a Music App Layout
- Responsive Design

Background

The Flexbox Layout (Flexible Box) module (a W3C Candidate Recommendation as of October 2017) **aims at providing a more efficient way to lay out, align and distribute space among items in a container**, **even when their size is unknown and/or dynamic** (thus the word “flex”).

- The **main idea behind the flex layout is to give the container the ability to alter its items’ width/height (and order) to best fill the available space** (mostly to accommodate to different types of display devices and screen sizes).

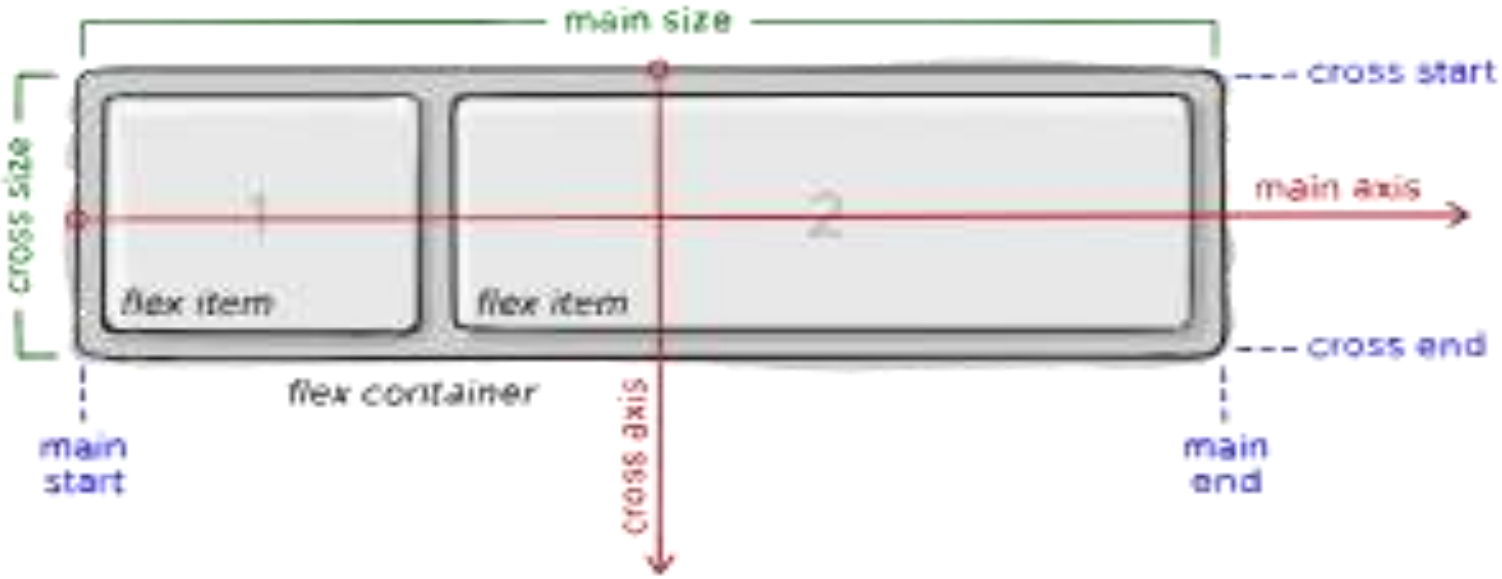
- A flex container expands items to fill available free space or shrinks them to prevent overflow.

Note: Flexbox layout is most appropriate to the components of an application, and small-scale layouts, while the Grid layout is intended for larger scale layouts.

Basics and terminology

- Since flexbox is a whole module and not a single property**, it involves a lot of things including its whole set of properties.**
  - **Parent element, known as "flex container")**
  - **Set on the Children (said "flex items").**

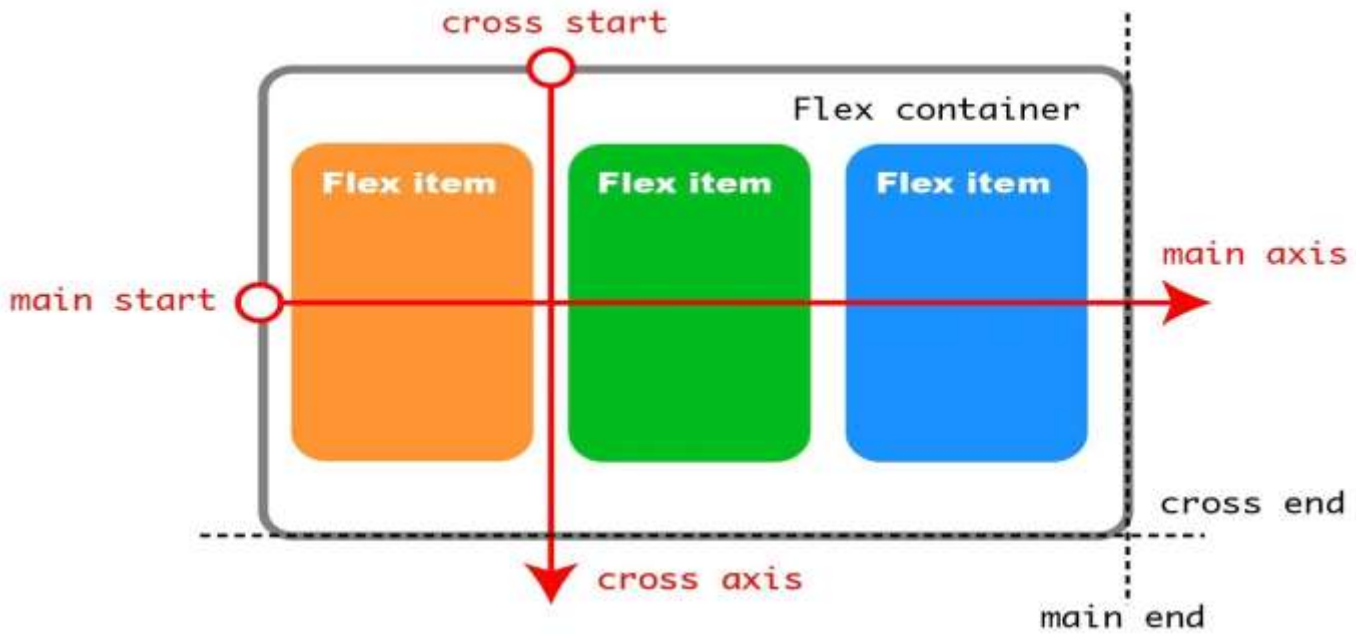
- If "regular" layout is based on both **block and inline flow directions**, the flex layout is based on "**flex-flow directions".**



Important Flexbox Terms

- **Flex container:** The parent element on which **display: flex** is applied.
- **Flex items:** The direct child elements inside the flex container.
- **Main axis:** The primary direction in which flex items are arranged.
- **Cross axis:** The axis perpendicular to the main axis.

For **flex-direction: row**, the main axis is horizontal and the cross axis is vertical.



How to Create a Flex Container

To use Flexbox, apply display: flex to the parent element.

CSS

```
.container {
        display: flex;
        background-color: lightgray;
    }
    .box {
        background-color: steelblue;
        color: white;
        padding: 20px;
        margin: 10px;
    }
```

HTML

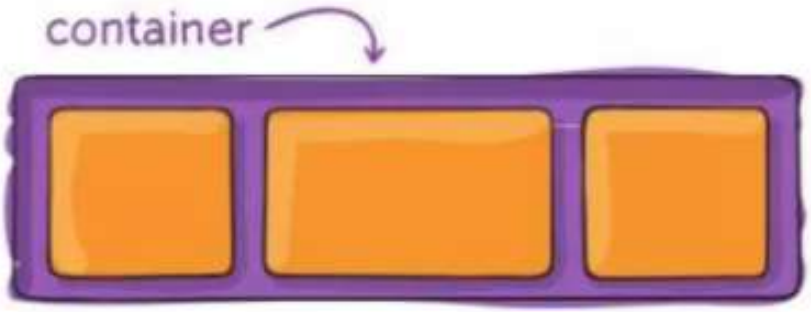
```
<div class="container">
    <div class="box">Box 1</div>
    <div class="box">Box 2</div>
    <div class="box">Box 3</div>
</div>
```

Output: Three boxes appear horizontally in one row.

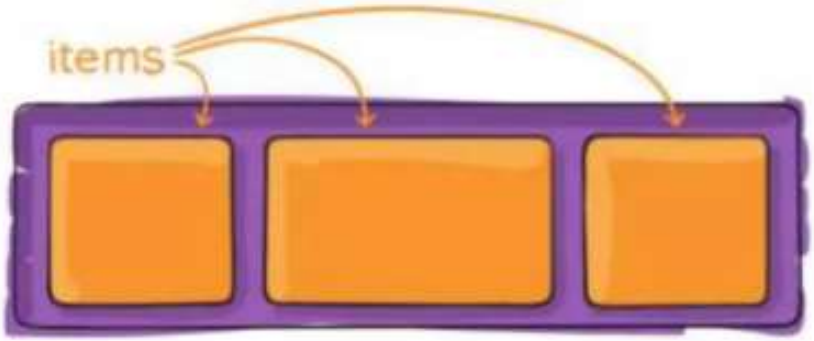


Illustration of three flex items arranged in a row.

Properties



Properties for the Parent
(flex container)



Properties for the Children
(flex items)

CHITKARA UNIVERSITY

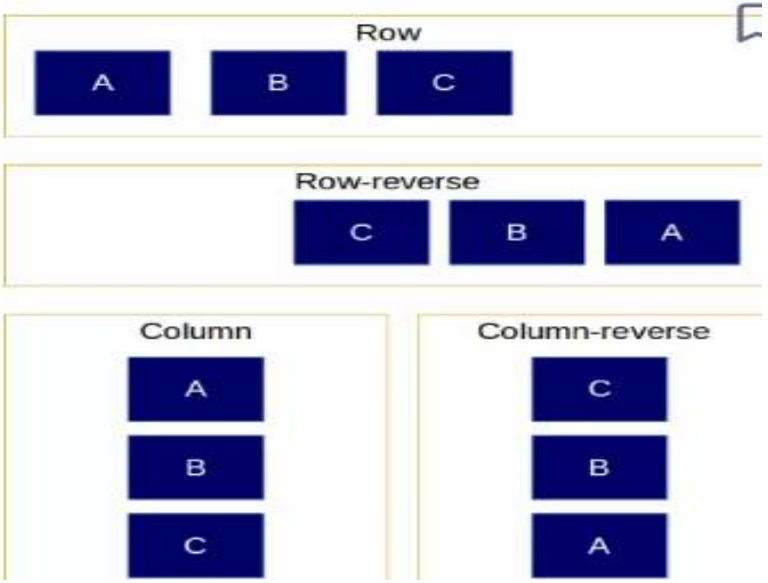
Flex Container Properties

flex-direction

It specifies the direction in which flex items are arranged.

- Render by x and y axis
- Axes defines flex-direction
  - Has four properties
    - Row
    - Row-reverse
    - Column
    - Column-reverse

Value	Description
row	Arranges items horizontally from left to right.
row-reverse	Arranges items horizontally in reverse order.
column	Arranges items vertically from top to bottom.
column-reverse	Arranges items vertically in reverse order.



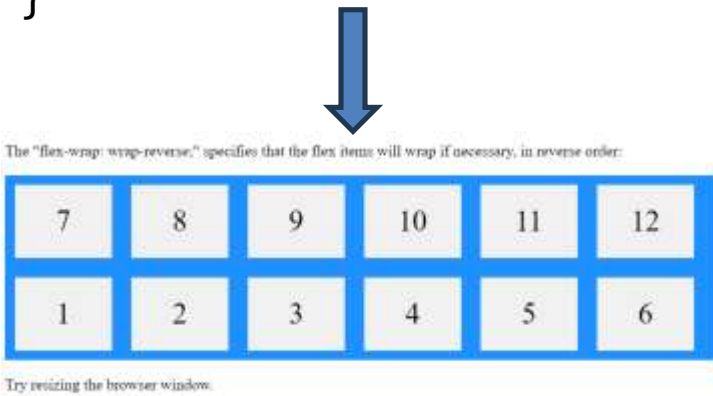

```
.flex-container {
  display: flex;
  flex-direction: column;
}
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
</div>
```

Flex wrap

Flex-wrap:nowrap|wrap|wrap-reverse

- It controls whether items stay on one line or move to another line when space is insufficient.
- Basically, the items in the container would try to fit in a single line
- Now this property is used to adjust the item as need.

```
.flex-container {
  display: flex;
  flex-wrap: wrap-reverse;
}
```



✓ **Nowrap (default) : All flex items will be on a line**

✓ **Wrap: used to wrap in multiple lines, from top to bottom**

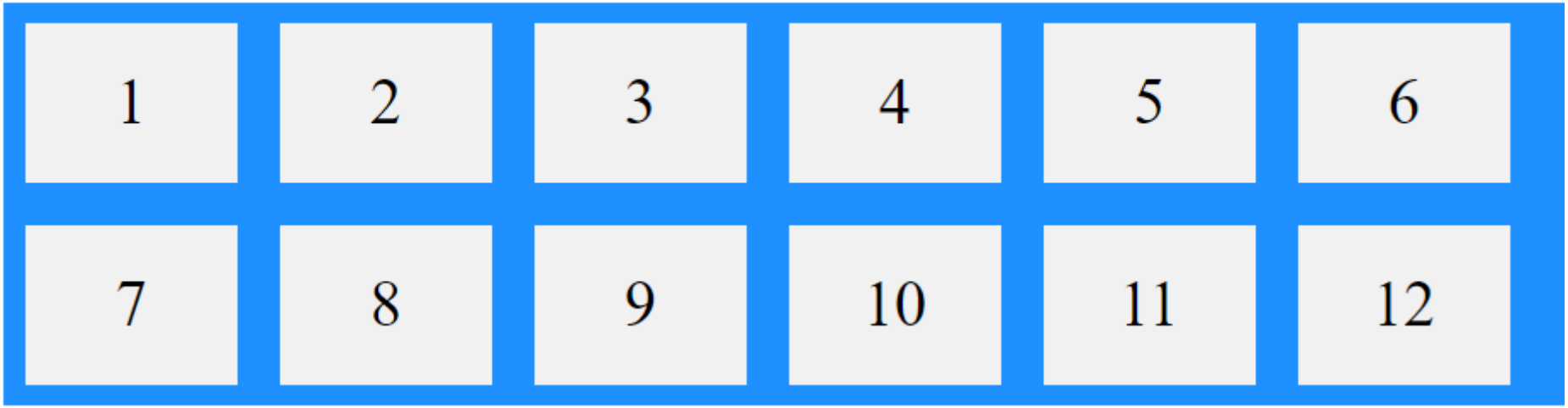
✓ **Wrap-reverse: reverse of Wrap property**

flex-flow

- The flex-flow property is a shorthand property for setting both the flex-direction and flex-wrap properties.

```
.flex-container {
  display: flex;
  flex-flow: row wrap;
}
```

The flex-flow property is a shorthand property for the flex-direction and the flex-wrap properties.



Aligning Items in flex-containers

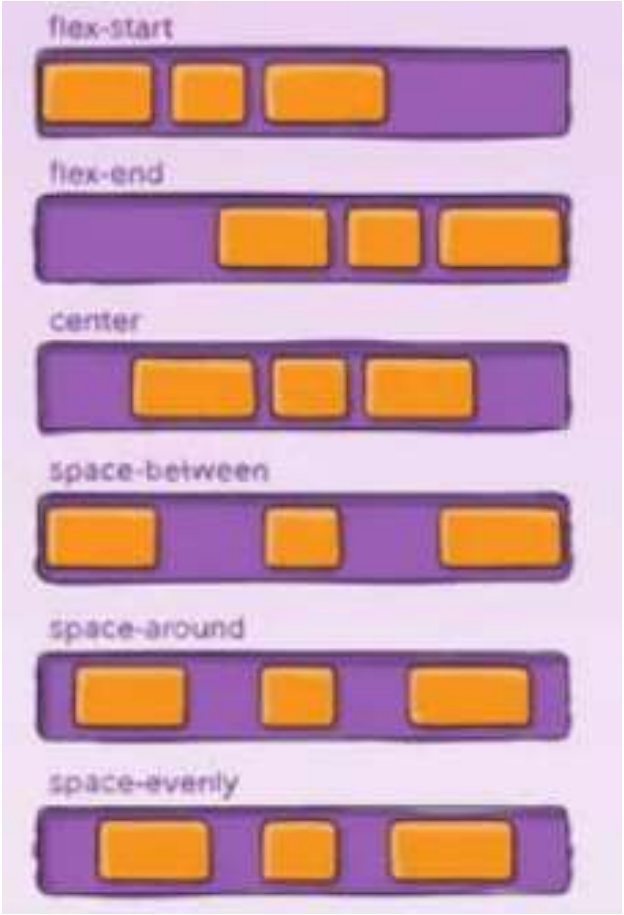
CHITKARA UNIVERSITY

- Flexbox quickly align the item present in the container.
- Alignment and distribution are controlled using the following properties:
  - justify-content
  - align-items
  - align-content

justify-content

- Aligns flex items along the main axis. For a row direction, this is generally the horizontal direction.

flex-start	Default value. Items are positioned at the beginning of the container
flex-end	Items are positioned at the end of the container
center	Items are positioned in the center of the container
space-between	Items will have space between them
space-around	Items will have space before, between, and after them
space-evenly	Items will have equal space around them
initial	Sets this property to its default value. Read about *initial*
inherit	Inherits this property from its parent element. Read about *inherit*



justify-content-Example

CHITKARA UNIVERSITY

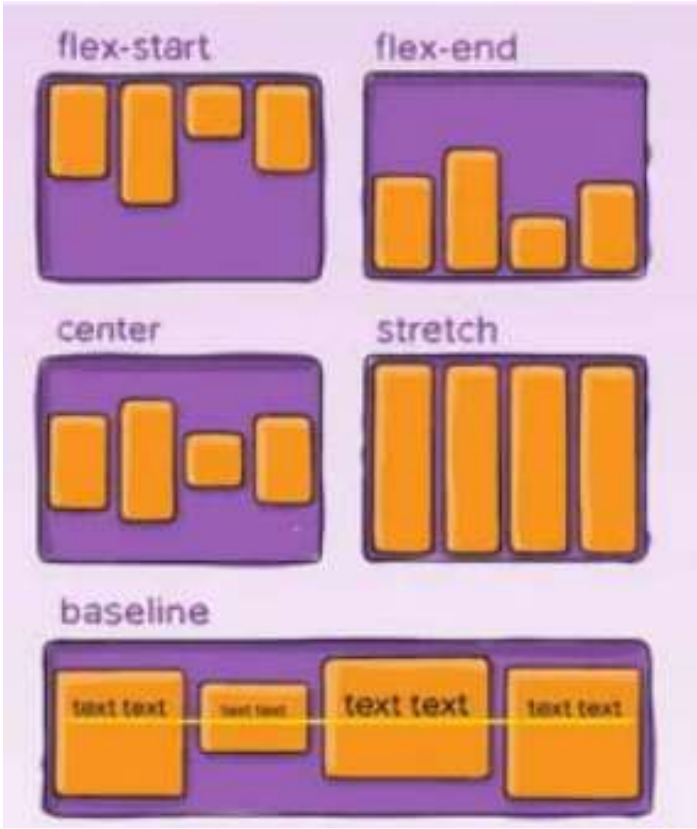
```
.flex-container {
  display: flex;
  justify-content: center;
}
```



Align Items

CHITKARA UNIVERSITY

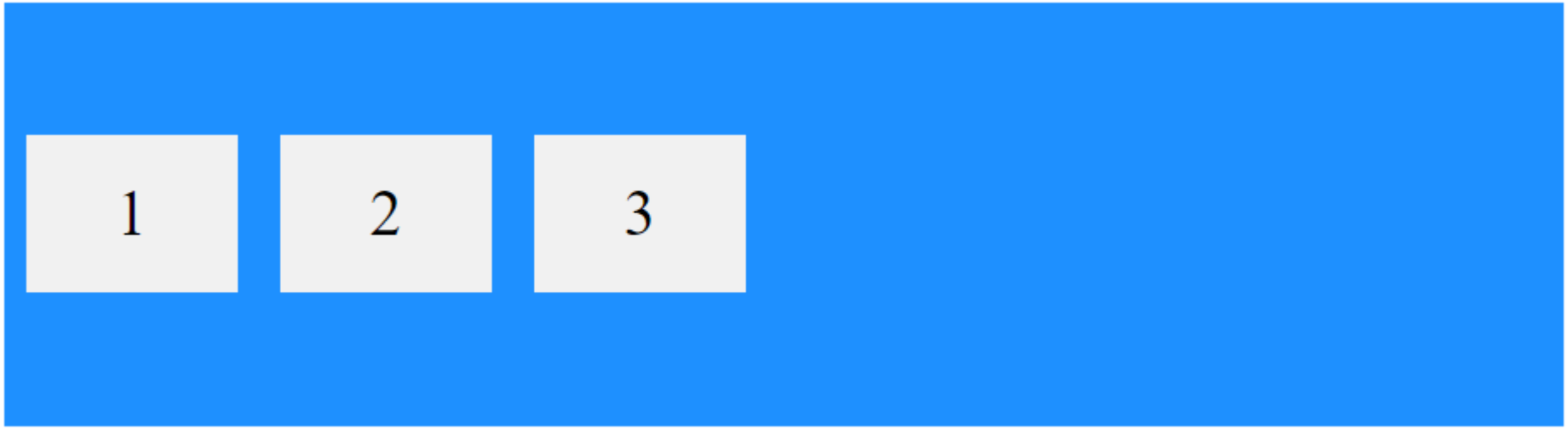
normal	Default. Behaves like 'stretch' for flexbox and grid items, or 'start' for grid items with a defined block size.
stretch	Items are stretched to fit the container
center	Items are positioned at the center of the container
flex-start	Items are positioned at the beginning of the container
flex-end	Items are positioned at the end of the container
start	Items are positioned at the beginning of their individual grid cells, in the block direction
end	Items are positioned at the end of the their individual grid cells, in the block direction
baseline	Items are positioned at the baseline of the container



Align Items-Example

```
.flex-container {
  display: flex;
  height: 200px;
  align-items: center;
}
```

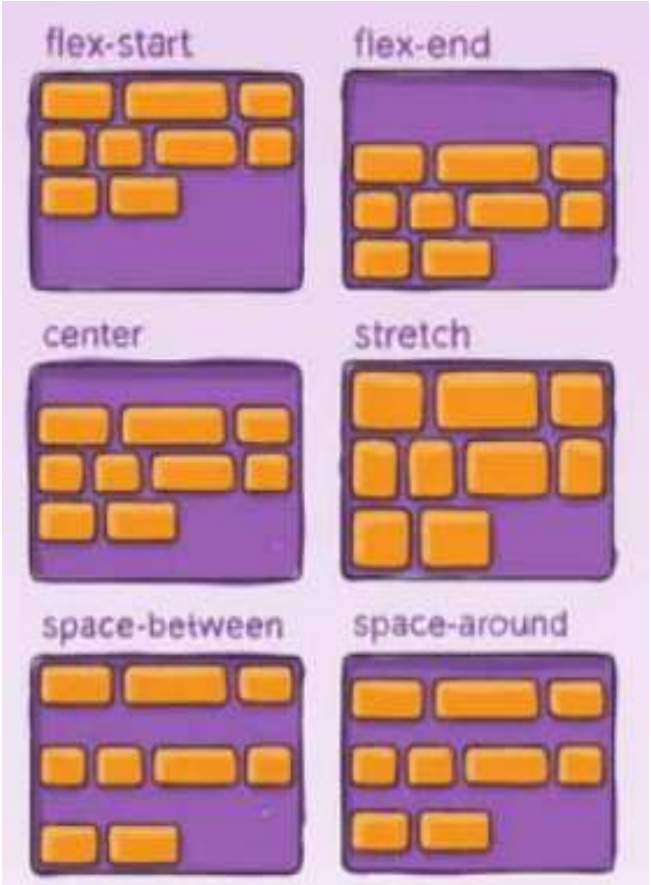
The "align-items: center;" aligns the flex items in the middle of the container:



Align Contents

- The align-content property specifies how flex lines are distributed along the cross axis in a flexbox container.

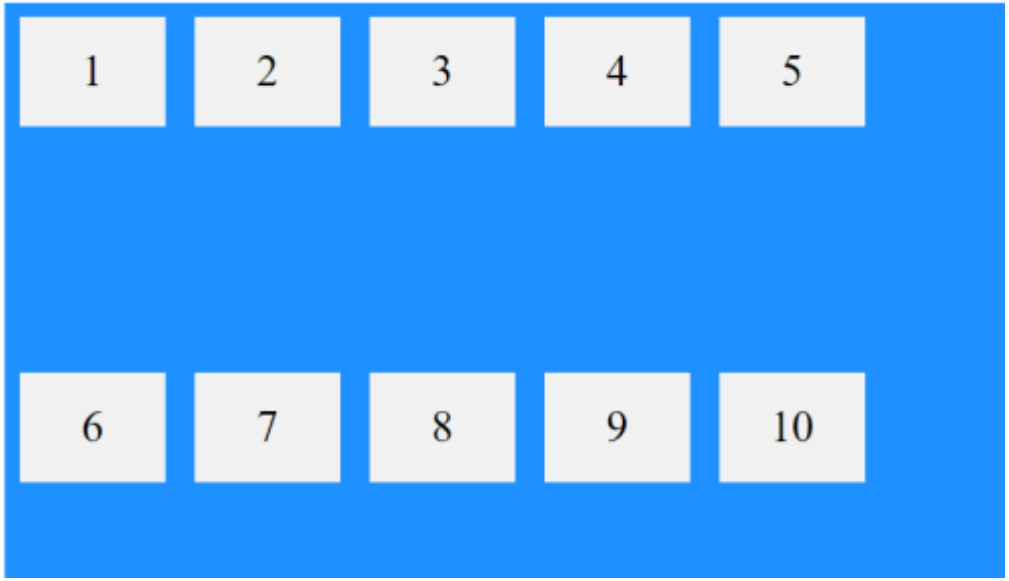
stretch	Default value. Lines stretch to take up the remaining space
center	Lines are packed toward the center of the flex container
flex-start	Lines are packed toward the start of the flex container
flex-end	Lines are packed toward the end of the flex container
space-between	Lines are evenly distributed in the flex container
space-around	Lines are evenly distributed in the flex container, with half-size spaces on either end



Align Contents-Example

- The align-content property specifies how flex lines are distributed along the cross axis in a flexbox container.

```
.flex-container {
  display: flex;
  height: 600px;
  flex-wrap: wrap;
  align-content: space-between;
}
```



CHITKARA UNIVERSITY

Child Elements (Items) Properties

Properties of Flex Items

CHITKARA UNIVERSITY

These properties are applied to the child elements.

Property	Purpose
order	Changes the visual order of an item.
flex-grow	Controls how much an item grows to use available space.
flex-shrink	Controls how much an item shrinks when space is insufficient.
flex	Shorthand for grow, shrink, and basis.

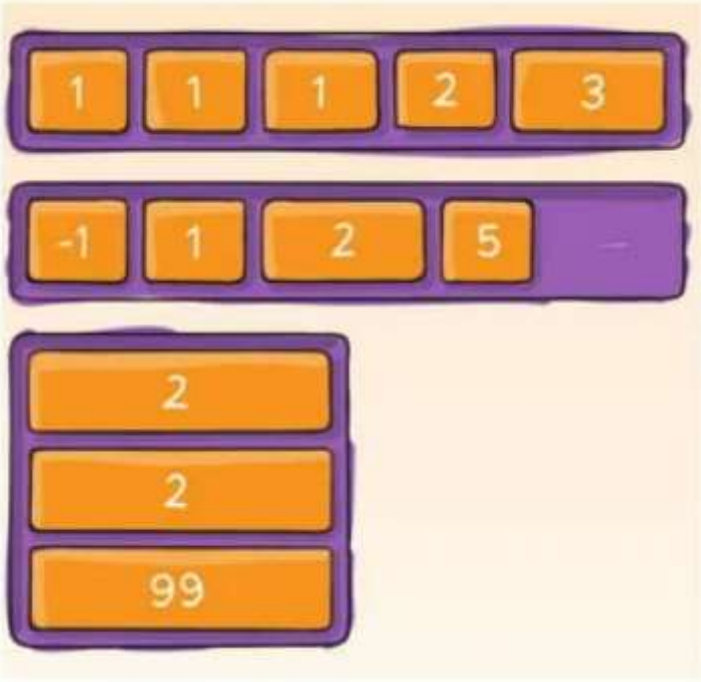
Order

CHITKARA UNIVERSITY

- **Order : <number>**

  - **CSS property sets the order to layout an items in flex grid**

  - **Items are sorted by ascending order value and then their source code order.**

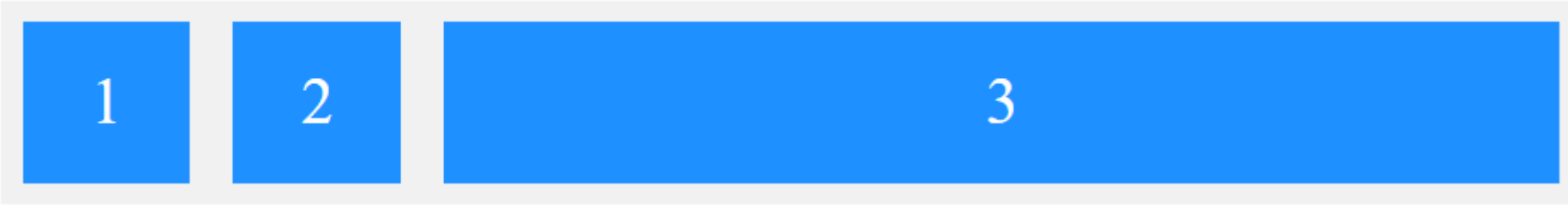



```
<div class="flex-container">
  <div style="order: 3">1</div>
  <div style="order: 2">2</div>
  <div style="order: 4">3</div>
  <div style="order: 1">4</div>
</div>
```

Flex Grow

- The **flex-grow property** specifies how much a flex item will grow relative to the rest of the flex items.
  - The value must be a number, default value is 0.

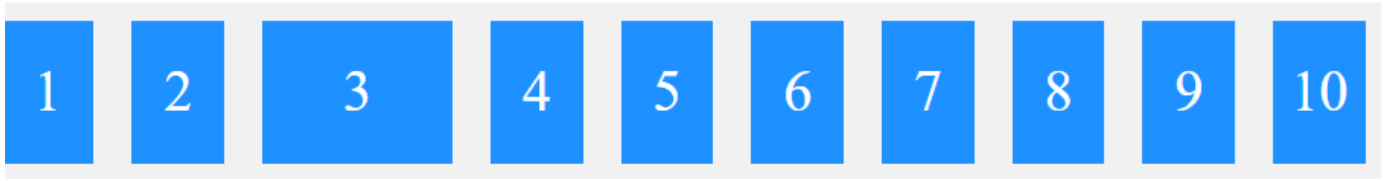
```
<div class="flex-container">
  <div style="flex-grow: 1">1</div>
  <div style="flex-grow: 1">2</div>
  <div style="flex-grow: 8">3</div>
</div>
```



Flex Shrink

- The **flex-shrink property** specifies how much a flex item will shrink relative to the rest of the flex items.
  - The value must be a number, default value is 1.

```
<div class="flex-container">
  <div>1</div>
  <div>2</div>
  <div style="flex-shrink: 0">3
  </div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
  <div>9</div>
  <div>10</div>
</div>
```



Responsive Flexbox

CHITKARA UNIVERSITY

- if you want to create a two-column layout for most screen sizes, and a one-column layout for small screen sizes (such as phones and tablets), you can change the flex-direction from row to column at a specific breakpoint (800px in the example below):

```
.flex-container {
  display: flex;
  flex-direction: row;
}

/* Responsive layout - makes a one column layout instead of
a two-column layout */
@media (max-width: 800px) {
  .flex-container {
    flex-direction: column;
  }
}
```

CHITKARA UNIVERSITY

Responsive Web Design(RWD)

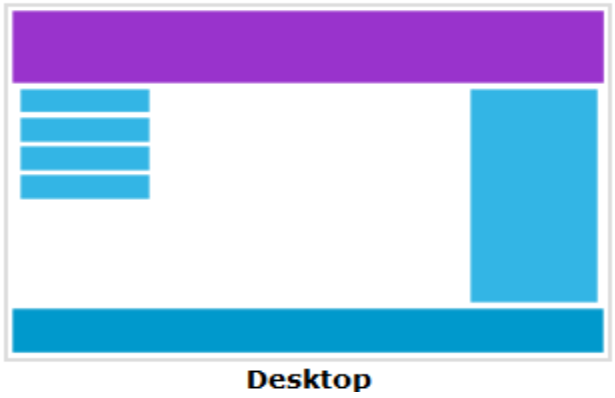
Responsive Web Design

CHITKARA UNIVERSITY

- Responsive web design makes your web page look good on all devices.
- Responsive Web Design is about using HTML and CSS to automatically resize, hide, shrink, or enlarge, a website, to make it look good on all devices (desktops, tablets, and phones)
- Responsive web design primarily uses HTML and CSS, and may also work with JavaScript and other web technologies.
- Responsive web design is a design approach, not a programming language.
- Responsive web design, or RWD, is a design approach that addresses the range of devices and device sizes, enabling automatic adaption to the screen, whether the content is viewed on a tablet, phone, television, or watch

Designing For The Best Experience For All Users

- Web pages can be viewed using many different devices: desktops, tablets, and phones. Your web page should look good, and be easy to use, regardless of the device.
- Web pages should not leave out information to fit smaller devices, but rather adapt its content to fit any device
- It is called responsive web design when you use CSS and HTML to resize, hide, shrink, enlarge, or move the content to make it look good on any screen.







Responsive Web Design - The Viewport

CHITKARA UNIVERSITY

- The viewport is the user's visible area of a web page.
- The viewport varies with the device, and will be smaller on a mobile phone than on a computer screen.
- Before tablets and mobile phones, web pages were designed only for computer screens, and it was common for web pages to have a static design and a fixed size.

Setting The Viewport

- HTML5 introduced a method to let web designers take control over the viewport, through the <meta> tag.
- You should include the following <meta> viewport element in all your web pages:

  <meta name="viewport" content="width=device-width, initial-scale=1.0">
- This gives the browser instructions on how to control the page's dimensions and scaling.
- The width=device-width part sets the width of the page to follow the screen-width of the device (which will vary depending on the device).
- The initial-scale=1.0 part sets the initial zoom level when the page is first loaded by the browser.



Without the viewport meta tag



With the viewport meta tag

Size Content to The Viewport

Users are used to scroll websites vertically on both desktop and mobile devices - but not horizontally! So, if the user is forced to scroll horizontally, or zoom out, to see the whole web page it results in a poor user experience.

- Some additional rules to follow:

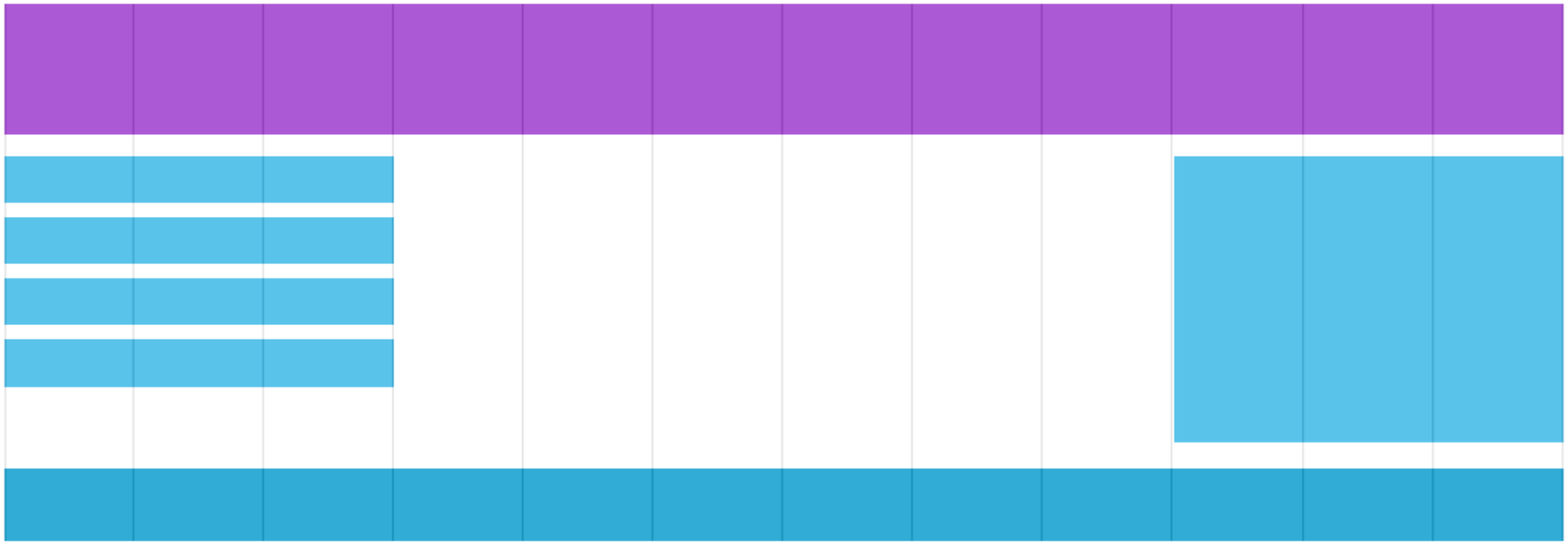
1. Do NOT use large fixed width elements - For example, if an image is displayed at a width wider than the viewport it can cause the viewport to scroll horizontally. Remember to adjust this content to fit within the width of the viewport.

2. Do NOT let the content rely on a particular viewport width to render well - Since screen dimensions and width in CSS pixels vary widely between devices, content should not rely on a particular viewport width to render well.

3. Use CSS media queries to apply different styling for small and large screens - Setting large absolute CSS widths for page elements will cause the element to be too wide for the viewport on a smaller device. Instead, consider using relative width values, such as width: 100%. Also, be careful of using large absolute positioning values. It may cause the element to fall outside the viewport on small devices.

Responsive Web Design - Grid-View

- Many web pages are based on a grid-view, which means that the page is divided into columns.
- Using a grid-view is very helpful when designing web pages. It makes it easier to place elements on the page.
- A responsive grid-view often has 12 columns, and has a total width of 100%, and will shrink and expand as you resize the browser window.



Building a Responsive Grid-View

CHITKARA UNIVERSITY

- First ensure that all HTML elements have the box-sizing property set to border-box. This makes sure that the padding and border are included in the total width and height of the elements.
- Add the following code in your CSS:

```
* {
  box-sizing: border-box;
}
```

- The following example shows a simple responsive web page, with two columns:

```
.menu {
  width: 25%;
  float: left;
}
.main {
  width: 75%;
  float: left;
}
```

Measurements Units

Measuring units – %, em, rem, px, vh, vw

1**. Percentage** (%):The percentage unit sets a value relative to another value defined by the relevant property, often the size of the containing element. This unit is highly flexible and allows for responsive design adjustments.

Example: Setting the font-size relative to the parent element's font-size.

font-size: 150%;

2. em: The em unit is relative to the font-size of the element it is used on. For example, 2em means two times the size of the current font.

Example: Setting the font-size relative to the parent element's font-size.

font-size: 2em;

Note: Here, 2em means 2 times the size of the current font.

3. rem: The rem unit is relative to the root element (usually the <html> element) font-size, making it consistent across the entire document.

Example: Setting the font-size relative to the browser base font-size.

font-size: 1.5rem

4. Pixels(px):

The pixel (px) is a CSS absolute length unit. It provides a predictable CSS measurement, although its physical size can vary across devices.

Example: Setting the font-size in pixels.

font-size: 16px;

5. Viewport Height (vh)

The vh unit is relative to 1% of the viewport's height, making it useful for setting sizes based on the height of the browser window.

Example: Setting the height relative to the viewport height.

height: 50vh;

6. Viewport Width (vw)

The vw unit is relative to 1% of the viewport's width, making it useful for setting sizes based on the width of the browser window.

Example: Setting the width relative to the viewport width.

width: 50vw;

Responsive Images

Responsive images are images that scale nicely to fit any browser size.

- **Using the width Property**

If the CSS width property is set to 100%, the image will be responsive and scale up and down:

<img src="img_girl.jpg" **style="width:100%;"**>

- **Using the max-width Property**

If the max-width property is set to 100%, the image will scale down if it has to, but never scale up to be larger than its original size:

<img src="img_girl.jpg" style="**max-width:100%;** height:auto;“ >

- **Different Images Depending on Browser Width**

The HTML <picture> element allows you to define different images for different browser window sizes.

Resize the browser window to see how the image below changes depending on the width:

```
<picture>
  <source srcset="img_smallflower.jpg" media="(max-width: 600px)">
  <source srcset="img_flowers.jpg" media="(max-width: 1500px)">
  <source srcset="flowers.jpg">
  <img src="img_smallflower.jpg" alt="Flowers">
</picture>
```



Responsive Text Size

- The text size can be set with a "vw" unit, which means the "viewport width“.That way the text size will follow the size of the browser window:
- **Example**
- <h1 style="**font-size:10vw**">Hello World</h1>



Media Queries

- Media query is a CSS technique introduced in CSS3.
- It uses the @media rule to include a block of CSS properties only if a certain condition is true.

Example

If the browser window is 600px or smaller, the background color will be lightblue:

```
@media only screen and (max-width: 600px)
{
  body {
    background-color: lightblue;
  }
}
```

Add a Breakpoint

When a web page is made with rows and columns, and it was responsive, but it did not look good on a small screen. Media queries can help with that. We can add a breakpoint where certain parts of the design will behave differently on each side of the breakpoint.



Desktop



Phone

Use a media query to add a breakpoint at 768px:

For Desktop

CHITKARA UNIVERSITY

When the screen (browser window) gets smaller than 768px, each column should have a width of 100%:

```
/* For desktop: */
.col-1 {width: 8.33%;}
.col-2 {width: 16.66%;}
.col-3 {width: 25%;}
.col-4 {width: 33.33%;}
.col-5 {width: 41.66%;}
.col-6 {width: 50%;}
.col-7 {width: 58.33%;}
.col-8 {width: 66.66%;}
.col-9 {width: 75%;}
.col-10 {width: 83.33%;}
.col-11 {width: 91.66%;}
.col-12 {width: 100%;}

@media only screen and (max-width: 768px) {
  /* For mobile phones: */
  [class*="col-"] {
    width: 100%;
  }
}
```

Always Design for Mobile First

CHITKARA UNIVERSITY

Mobile First means designing for mobile before designing for desktop or any other device (This will make the page display faster on smaller devices).This means that we must make some changes in our CSS. Instead of changing styles when the width gets *smaller* than 768px, we should change the design when the width gets *larger* than 768px. This will make our design Mobile First:

For Mobile Phone

CHITKARA UNIVERSITY

```
/* For mobile phones: */
[class*="col-"] {
  width: 100%;
}

@media only screen and (min-width: 768px) {
  /* For desktop: */
  .col-1 {width: 8.33%;}
  .col-2 {width: 16.66%;}
  .col-3 {width: 25%;}
  .col-4 {width: 33.33%;}
  .col-5 {width: 41.66%;}
  .col-6 {width: 50%;}
  .col-7 {width: 58.33%;}
  .col-8 {width: 66.66%;}
  .col-9 {width: 75%;}
  .col-10 {width: 83.33%;}
  .col-11 {width: 91.66%;}
  .col-12 {width: 100%;}
}
```

Orientation: Portrait / Landscape

CHITKARA UNIVERSITY

- Media queries can also be used to change layout of a page depending on the orientation of the browser. We can have a set of CSS properties that will only apply when the browser window is wider than its height, a so called "Landscape" orientation:

Example

The web page will have a lightblue background if the orientation is in landscape mode:

```
@media only screen and (orientation: landscape) {
  body {
    background-color: lightblue;
  }
}
```

Hide Elements With Media Queries

Another common use of media queries, is to hide elements on different screen sizes:

It will be hidden on small screens.

- **Example**

```
/* If the screen size is 600px wide or less, hide the element */
@media only screen and (max-width: 600px) {
  div.example {
    display: none;
  }
}
```

Change Font Size With Media Queries

CHITKARA UNIVERSITY

We can also use media queries to change the font size of an element on different screen sizes:

- **Variable Font Size.**

Example

```
/* If the screen size is 601px or more, set the font-size of <div> to 80px */
@media only screen and (min-width: 601px) {
  div.example {
    font-size: 80px;
  }
}
/* If the screen size is 600px or less, set the font-size of <div> to 30px */
@media only screen and (max-width: 600px) {
  div.example {
    font-size: 30px;
  }
}
```

CHITKARA UNIVERSITY

Animations and 3D Space: Transitions and Transforms

Introduction

CHITKARA UNIVERSITY

CSS provides visual effects that make web pages interactive and attractive.

- **Transform:** Changes the position, size, rotation, or shape of an element.
- **Transition:** Changes a CSS property smoothly over time.
- **Animation:** Changes an element's styles through multiple stages over time.
- **3D space:** Allows elements to rotate or move along the X, Y, and Z axes.

What is a transform?

--"to change in form, appearance, or structure; metamorphose"

--“The CSS transform property lets you modify the coordinate space of the CSS visual formatting model. Using it, elements can be translated, rotated, scaled, and skewed according to the values set.”

Transforms: Syntax

No Transforms

transform: none;

One Transform

transform: transform-function;

Multiple Transforms

transform: transform-function transform-function transform-function;

Transform Functions

- Rotate
- Scale
- Skew
- Translate
- ... and Matrix

Rotate

• Rotates an element clockwise from its current position. Counterclockwise rotations can be achieved by using negative values.

Rotate: Syntax

transform: rotate(angle);

* angles are expressed as (5deg) or (-5deg), for example.

Examples

Box


```
<div class="box"></div>
```

```
.box {
  width: 3em;
  height: 3em;
  background: #bced91;
}
```

Rotate: Examples

Rotate 25 Degrees to the Right

```
.box {
  transform: rotate(25deg);
}
```

Rotate 25 Degrees to the Left

```
.box {
  transform: rotate(-25deg);
}
```

- With the CSS transform property you can use the following 3D transformation methods:
  - rotateX()
  - rotateY()
  - rotateZ()

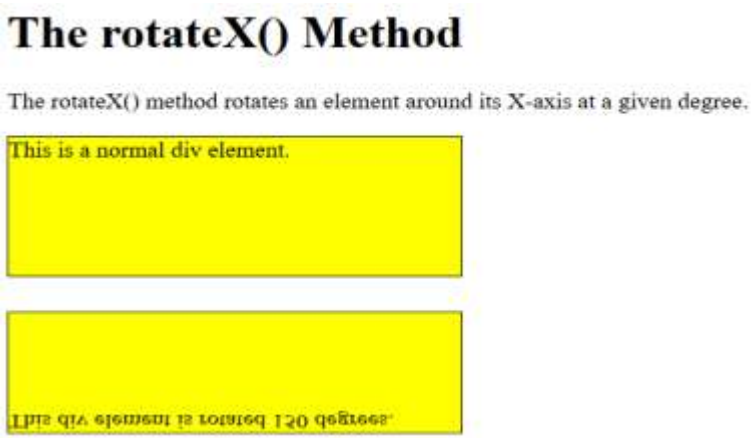
The rotateX() Method

The rotateX() method rotates an element around its X-axis at a given degree:

Example

```
#myDiv {
  transform: rotateX(150deg);
}
```

Output Given below:




```
div {
  width: 300px;
  height: 100px;
  background-color: yellow;
  border: 1px solid black;
}

#myDiv {
  transform: rotateX(150deg);
}
</style>
</head>
<body>

<h1>The rotateX() Method</h1>

<p>The rotateX() method rotates an element around its X-axis at a given degree.</p>

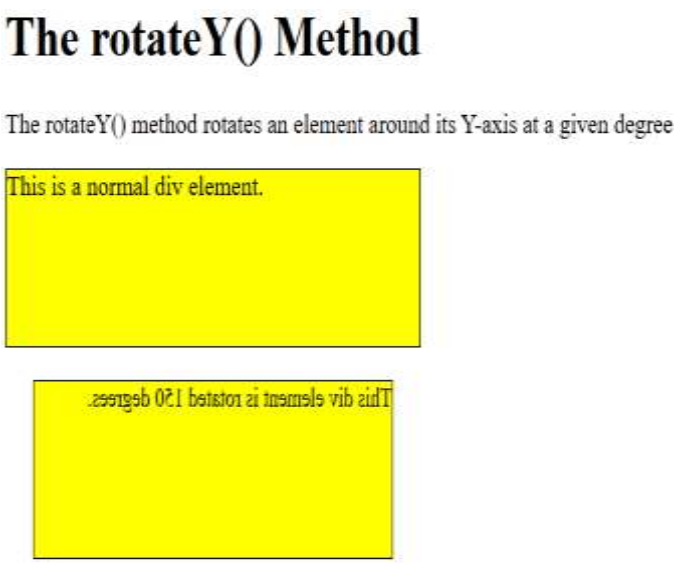
<div>
This a normal div element.
</div>

<div id="myDiv">
This div element is rotated 150 degrees.
</div>

</body>
</html>
```

The rotateY() Method
The rotateY() method rotates an element around its Y-axis at a given degree:

Example
#myDiv {
  transform: rotateY(150deg);
} Output is given below:




```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 300px;
  height: 100px;
  background-color: yellow;
  border: 1px solid black;
}

#myDiv {
  transform: rotateY(150deg);
}
</style>
</head>
<body>

<h1>The rotateY() Method</h1>

<p>The rotateY() method rotates an element around its Y-axis at a given degree.</p>

<div>
This a normal div element.
</div>

<div id="myDiv">
This div element is rotated 150 degrees.
</div>

</body>
</html>
```

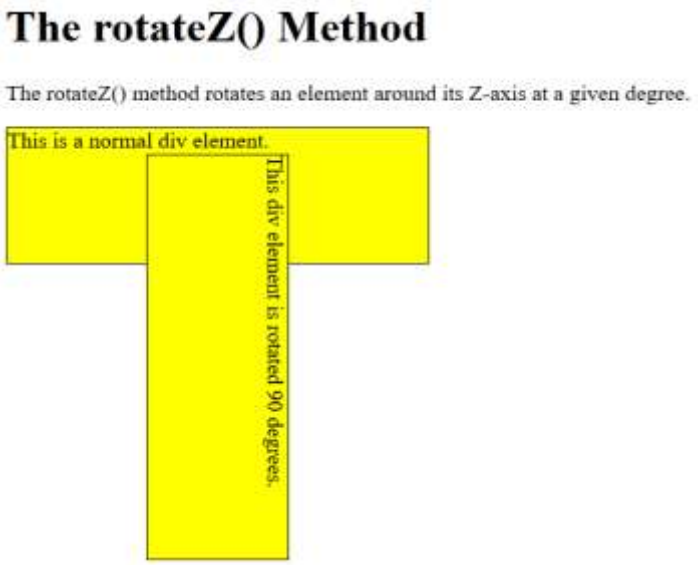
The rotateZ() Method

The rotateZ() method rotates an element around its Z-axis at a given degree:

Example

```
#myDiv {
  transform: rotateZ(90deg);
}
```

Out is given below:




```
<!DOCTYPE html>
<html>
<head>
<style>
div {
  width: 300px;
  height: 100px;
  background-color: yellow;
  border: 1px solid black;
}

#myDiv {
  transform: rotateZ(90deg);
}
</style>
</head>
<body>

<h1>The rotateZ() Method</h1>

<p>The rotateZ() method rotates an element around its Z-axis at a given degree.</p>

<div>
This a normal div element.
</div>

<div id="myDiv">
This div element is rotated 90 degrees.
</div>

</body>
</html>
```

Scale

- Scales an element relative to its original size. The values specified within scale transforms are unitless and should be thought of as a multiplier.

Scale: Syntax

X and Y axis Scale

```
transform:      scale(valueX,  valueY);
```

Just X axis Scale

```
transform:  scaleX(valueX);
```

Just Y axis Scale

```
transform:  scaleY(valueY);
```

Scale: Examples

CHITKARA UNIVERSITY

Shrink to half size



```
.box {
    transform: scale(0.5);
}
```

Double Y Axis



```
.box {
  transform: scale(1, 2);
}
```

Skew

- Skews an element along the X and/or Y axis by an angle value.

Skew: Syntax

X and Y axis Skew

```
transform: skew(angleX, angleY);
```

* do not use

Just X axis Skew

```
transform: skewX(angleX);
```

Just Y axis Skew

```
transform: skewY(angleY);
```

Skew: Examples

CHITKARA UNIVERSITY

Skew left



```
.box {
  transform: skewX(25deg);
}
```

Skew left and up



```
.box {
  transform: skewX(25deg) skewY(25deg);
}
```

Translate

Moves an element vertically and/or horizontally relative to its current position. Positive values will move the element right on the X axis or down on the Y axis. Conversely, negative values will move the element left on the X axis or up on the Y axis. Any regular length value can be used (px, em, etc.).

Translate: Syntax

X and Y axis translation

```
transform:        translate(valueX, valueY);
```

Just X axis translation

```
transform:        translateX(valueX);
```

Just Y axis translation

```
transform:        translateY(valueY);
```

Translate: Example



Starting Point

```
.box-wrap {
   width: 6em;
   height:
   6em;
   border: 5px solid #282828;
}
```



Move Box to Bottom Right

```
.box {
   transform: translate(3em, 3em);
}
```

Why would you use translate when you can use more established CSS standards for layout and formatting?

- Think of translate as a more efficient way to move placement in a transition or animation.

Transform Origin

A CSS property that defines the point from which a CSS transform is applied to an element. By default, transform-origin is set to the center of an element (50% 50%).

Transform Origin Syntax

```
transform-origin: x-offset y-offset;
```

x-offset: a length or percentage from the left edge of the element's box.

y-offset: a length or percentage from the top edge of the element's box.

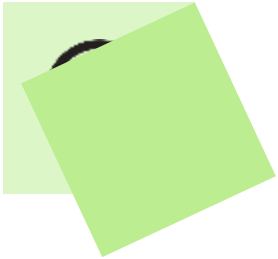
Accepted keywords

- left (0%)
- center (50%)
- right (100%)
- top (0%)
- bottom (100%)

Transform Origin: Examples

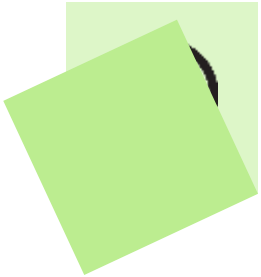
CHITKARA UNIVERSITY

Rotate 25 Degrees to the Left & Transform Origin at Top Right



```
.box {
  transform: rotate(-25deg);
  transform-origin: 100% 0%;
}
```

Rotate 25 Degrees to the Left & Transform Origin at Bottom Right



```
.box {
  transform: rotate(-25deg);
  transform-origin: right bottom;
}
```

CSS Transform Properties

The following table lists all the 3D transform properties:

Property	Description
transform	Applies a 2D or 3D transformation to an element
transform-origin	Allows you to change the position on transformed elements
transform-style	Specifies how nested elements are rendered in 3D space
perspective	Specifies the perspective on how 3D elements are viewed
perspective-origin	Specifies the bottom position of 3D elements
backface-visibility	Defines whether or not an element should be visible when not facing the screen

CSS 3D Transform Methods

CHITKARA UNIVERSITY

Function	Description
matrix3d (*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*,*n*)	Defines a 3D transformation, using a 4x4 matrix of 16 values
translate3d(*x*,*y*,*z*)	Defines a 3D translation
translateX(*x*)	Defines a 3D translation, using only the value for the X-axis
translateY(*y*)	Defines a 3D translation, using only the value for the Y-axis
translateZ(*z*)	Defines a 3D translation, using only the value for the Z-axis
scale3d(*x*,*y*,*z*)	Defines a 3D scale transformation
scaleX(*x*)	Defines a 3D scale transformation by giving a value for the X-axis
scaleY(*y*)	Defines a 3D scale transformation by giving a value for the Y-axis
scaleZ(*z*)	Defines a 3D scale transformation by giving a value for the Z-axis
rotate3d(*x*,*y*,*z*,*angle*)	Defines a 3D rotation
rotateX(*angle*)	Defines a 3D rotation along the X-axis
rotateY(*angle*)	Defines a 3D rotation along the Y-axis
rotateZ(*angle*)	Defines a 3D rotation along the Z-axis
perspective(*n*)	Defines a perspective view for a 3D transformed element

What is a transition?

"movement, passage, or change from one position, state, stage, subject, concept, etc., to another; change"

“The transition property is a shorthand property used to represent up to four transition-related longhand properties [that] allow elements to change values over a specified duration, animating the property changes, rather than having them occur immediately.”

Transition Properties

- transition-property
- transition-duration
- transition-timing
- function transition-delay

Transitions: Syntax

CHITKARA UNIVERSITY

Shorthand

```
transition: property duration timing-function delay;
```

Longhand

```
transition-property: property;
transition-duration: duration;
transition-timing-function: timing-function;
transition-delay: delay;
```

Transition Property

- Specifies which property or properties will be impacted by the transition.

Syntax

```
transition-property: value;
```

Accepted Values

- all (default)
- none
- single property name
- multiple property names, separated by commas

Transitions: Example Stage 1

CHITKARA UNIVERSITY

Transition Property Only

Starting State



```
.box {
  transition-property: color, background-color,
  width; color: #000000;
  background-color:
  #bced91; width: 3em;
}
```

Active State



```
.box:hover { color:
  #FFFFFF;
  background-color: #4f94e2;
  width: 3.6em;
}
```

Transition Duration Property

- Specifies the amount of time the transition will take.

Syntax

```
transition-duration: value;
```

Accepted Values

- time in seconds (s) or milliseconds (ms)
- multiple time values separated by commas
- initial
- inherit

Transitions: Example Stage 2

Transition Property + Duration

Starting State



```
.box {
  transition-property: color, background-color, width;
  transition-duration: 2s;
  color: #000000;
  background-color: #bced91;
  width: 3em;
}
```

Active State



```
.box:hover {
  color: #FFFFFF;
  background-color: #4f94e2;
  width: 3.6em;
}
```

Transition Timing Property

- Specifies the pace at which the transition will run.

Syntax

```
transition-timing-function:    value;
```

- ease
- linear
- ease-in
- ease-out
- ease-in-out

Accepted Values

- step-start step-end
- custom cubic Bézier curve value
- custom stepping function value

Transitions: Example Stage 3

CHITKARA UNIVERSITY

Transition Property + Duration + Timing

Starting State



```
.box {
  transition-property: color, background-color, width;
  transition-duration: 2s;
  transition-timing-function: ease-in;
  color: #000000;
  background-color: #bced91;
  width: 3em;
}
```

Active State



```
.box:hover {
  color: #ffffff;
  background-color: #4f94e2;
  width: 3.6em;
}
```

Transition Delay Property

- Specifies when the transition will start relative to the instant the property value changes.

Syntax

```
transition-delay: value;
```

Accepted Values

- time in seconds (s) or milliseconds (ms)
- multiple time values separated by commas
- initial
- inherit

Transitions: Example Stage 4

Transition Property + Duration + Timing + Delay

Starting State



```
.box {
  transition-property: color, background-color, width;
  transition-duration: 2s;
  transition-timing-function: ease-in;
  transition-delay: 3s;
  color: #000000;
  background-color: #bced91;
  width: 3em;
}
```

Active State



```
.box:hover {
  color: #FFFFFF;
  background-color: #4f94e2;
  width: 3.6em;
}
```

Animations

- CSS animations allow an element to change styles through multiple stages without requiring a hover action or other user interaction.
- Animations use the @keyframes rule to define the stages of an effect.

Syntax

```
@keyframes animationName {
    from {
        /* Starting styles */
    }
    to {
        /* Ending styles */
    }
}
```

The animation is then applied to an element using the animation property.

Real-life examples of CSS Animation

CHITKARA UNIVERSITY

- Moving car — transportation websites
- Loading spinner — websites/apps while data loads
- Notification bell — alert animation
- Like button — social media
- Add-to-cart animation — shopping websites
- Rain falling — weather websites
- Sliding menu — mobile websites
- Image/banner slider — homepages

Animations: Keyframes

CHITKARA UNIVERSITY

- Keyframes establish the behavior of an animation over time.

Required Properties

- animation-duration
- animation-name

Optional Properties

- animation-timing-function
- animation-delay
- animation-iteration-count
- animation-direction
- animation-fill-mode
- animation-play-state

Animation: Animation Duration Property

- Specifies the length of time one cycle of the animation will take.

Syntax

```
animation-duration: value;
```

Accepted Values

- Time in seconds (s) or milliseconds (ms)

Animations: Animation Name Property

- Specifies the animation/s that should be applied to the CSS element.

Syntax

```
animation-name: [custom-value];
```

Accepted values

- Custom string with no spaces

Animations: Animation Iteration Count Property

- Specifies the number of times the animation cycle should run.

Syntax

```
animation-iteration-count: value;
```

Accepted values

- a number
- infinite
- initial
- inherit

Animation: Syntax to Call Animation

```
.element {
  animation-duration:
  value; animation-name:
  custom-value;
}
```

*Optional animation properties would be included in the same code block as animation-duration and animation-name.

Animation: Keyframes Syntax

```
@keyframes animation-name {
  keyframe-location {
    css-properties;
  }
  keyframe-location {
   css-properties;
  }
}
```

animation name: the animation-name string from your CSS

keyframe location: waypoint of the animation

- Can either be percentages between 0-100% or in the case of animations with only two waypoints, the keywords “from” and “to.”

CSS properties: the CSS properties that change throughout the animation. Not all CSS properties are “animatable.”

Animations: Example



```
.box {
  animation-duration: 1s; animation-
  name: zoom-pulse;
  animation-iteration-count: infinite;
}

@keyframes zoom-pulse {
  0%   { transform: rotate(-9deg) scale(0.8); }
  17%  { transform: rotate(-3deg) scale(1); }
  34%  { transform: rotate(3deg) scale(1.2); }
  51%  { transform: rotate(9deg) scale(1.4); }
  68%  { transform: rotate(3deg) scale(1.2); }
  85%  { transform: rotate(-3deg) scale(1); }
  100% { transform: rotate(-9deg) scale(0.8); }
}
```

Animations: Default Animation Property Values

```
animation-name: none;
animation-duration: 0s;
animation-timing-function: ease;
animation-delay: 0s;
animation-iteration-count: 1;
animation-direction: normal;
animation-fill-mode: none;
animation-play-state: running;
```

Working with Multiple Animations

- You can apply multiple animations to a single element by separating the animation names and properties with commas.

Example:

```
@keyframes grow {
   from { transform: scale(1); }
   to { transform: scale(1.5); }
}
@keyframes fade {
   from { opacity: 1; }
   to { opacity: 0.5; }
}
.multi-animated-box {
   width: 100px;
   height: 100px;
   background-color: purple;
   animation: grow 2s infinite alternate, fade 4s infinite alternate;
}
```

This example scales the box up and down while simultaneously fading it in and out.

Animate.css

- Animate.css is a library of pre-built CSS animations that are easy to use.

Usage:

- Include Animate.css in your project:

```
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css"/
>
```

- Apply an animation class to an element:

```
<div class="animate__animated animate__bounce">
    I will bounce!
</div>
```

Example:

- **Bounce Animation:**

```
<div class="animate__animated animate__bounce">
    This text will bounce.
</div>
```

CSS 3D Space

3D transforms allow elements to move and rotate along three axes
The three axes
•**X-axis:** Horizontal direction.
•**Y-axis:** Vertical direction.
•**Z-axis:** Depth direction, toward or away from the viewer.

Important 3D transform functions

Function	Description
rotateX()	Rotates around the X axis.
rotateY()	Rotates around the Y axis.
rotateZ()	Rotates around the Z axis.
translateZ()	Moves an element along the Z axis.
translate3d()	Moves an element along all three axes.
scale3d()	Scales an element along all three axes.
perspective()	Defines the viewer's perspective and the appearance of depth.

Example: 3D rotation

CSS

```
.container {
    perspective: 500px;
    margin: 50px;
}
.box {
    width: 150px;
    height: 100px;
    background-color: teal;
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: transform 1s;
}
.box:hover {
    transform: rotateY(45deg);
}
```

HTML

```
<div class="container">
        <div class="box">3D Box
</div>
   </div>
```

Output: The box rotates around the Y axis when the mouse pointer moves over it. The perspective creates a sense of depth.

3D Cube Animation

CHITKARA UNIVERSITY

- A 3D cube animation can be created using CSS to rotate multiple sides of a cube.

Example:

```
.cube {
  width: 100px;
  height: 100px;
  position: relative;
  transform-style: preserve-3d;
  animation: spin 5s infinite linear;
}
.face {
  position: absolute;
  width: 100px;
  height: 100px;
  background-color: rgba(255, 255, 255, 0.9);
  border: 1px solid #ccc;
}
.face1 { transform: rotateY(0deg) translateZ(50px); }
.face2 { transform: rotateY(90deg) translateZ(50px); }
.face3 { transform: rotateY(180deg) translateZ(50px); }
.face4 { transform: rotateY(-90deg) translateZ(50px); }

@keyframes spin {
  from { transform: rotateY(0deg); }
  to { transform: rotateY(360deg); }
}
```

This code creates a rotating 3D cube with four visible faces.

Building the Second Face of the Cube

CHITKARA UNIVERSITY

- **Adding More Faces:**
  - To create additional faces, simply add more .face elements and use CSS transforms to position them in 3D space.

  Example:

  ```
  .face5 { transform: rotateX(90deg) translateZ(50px); }
  .face6 { transform: rotateX(-90deg) translateZ(50px); }
  ```

- This completes the cube by adding the top and bottom faces, creating a full 3D object.

CHITKARA UNIVERSITY

Pseudo Classes and Pseudo Elements

CSS Pseudo-classes

CHITKARA UNIVERSITY

What are Pseudo-classes?

A pseudo-class is used to define a special state of an element.

For example, it can be used to:

- Style an element when a user mouses over it
- Style visited and unvisited links differently
- Style an element when it gets focus

Syntax

The syntax of pseudo-classes:

```
selector:pseudo-class {
  property: value;
}
```

Anchor Pseudo-classes

CHITKARA UNIVERSITY

```
<!DOCTYPE html>
<html>
<head>
<style>
/* unvisited link */
a:link {
  color: red;
}
/* visited link */
a:visited {
  color: green;
}
/* mouse over link */
a:hover {
  color: hotpink;
}
```

```
/* selected link */
a:active {
  color: blue;
}
</style>
</head>
<body>

<h2>Styling a link depending on state</h2>

<p><b><a href="default.asp" target="_blank">This is a link</a></b></p>
<p><b>Note:</b> a:hover MUST come after a:link and a:visited in the CSS definition in order to
      be effective.</p>
<p><b>Note:</b> a:active MUST come after a:hover in the CSS definition in order to be
      effective.</p>

</body>
</html>
```

Output

CHITKARA UNIVERSITY

Styling a link depending on state

This is a link

Note: a:hover MUST come after a:link and a:visited in the CSS definition in order to be effective.

Note: a:active MUST come after a:hover in the CSS definition in order to be effective.

CSS - The :first-child Pseudo-class

CHITKARA UNIVERSITY

The :first-child pseudo-class matches a specified element that is the first child of another element.

Match the first <p> element

In the following example, the selector matches any <p> element that is the first child of any element:

Example

```
p:first-child {
  color: blue;
}
```

Code

```
<!DOCTYPE html>
<html>

<head>  <style>
p:first-child {
  color: blue;
}
</style></head>
<body>
<p>This is some text.</p>
<p>This is some text.</p>
<div>
  <p>This is some text.</p>
  <p>This is some text.</p>
</div>
</body>
</html>
```

Output

CHITKARA UNIVERSITY

This is some text.

This is some text.

This is some text.

This is some text.

CSS - The :lang Pseudo-class

The `:lang` pseudo-class allows you to define special rules for different languages.

In the example below, `:lang` defines the quotation marks for <q> elements with lang="no":

```
<!DOCTYPE html>
<html>
<head>
<style>
q:lang(no) {
  quotes: "~" "~";
}
</style>
</head>
<body>

<p>Some text <q lang="no">A quote in a paragraph</q> Some text.</p>
<p>In this example, :lang defines the quotation marks for q elements with
lang="no":</p>

</body>
</html>
```

Output

Some text ~A quote in a paragraph~ Some text.

In this example, :lang defines the quotation marks for q elements with lang="no":

CSS Pseudo-elements

CHITKARA UNIVERSITY

What are Pseudo-Elements?

A CSS pseudo-element is used to style specified parts of an element.

For example, it can be used to:

- Style the first letter, or line, of an element
- Insert content before, or after, the content of an element

Syntax

The syntax of pseudo-elements:

```
selector::pseudo-element {
  property: value;
}
```

1. ::first-letter

```
<style>
    p::first-letter {
        color: red;
        font-size: 35px;
        font-weight: bold;
    }
</style>

<p>Welcome to CSS programming.</p>
```

Output: The letter W appears larger, bold, and red.

Expected output

Welcome to CSS programming.

2. ::first-line

```
<style>
    p::first-line {
        color: blue;
        font-weight: bold;
    }
</style>
<p>
    CSS is used to style web pages.
    It controls colors, fonts, spacing and layout.
</p>
```

Output: The first displayed line of the paragraph appears blue and bold. The exact line depends on the available width.

3. ::before

This pseudo-element inserts generated content before the element's existing content.

```
<style>
    p::before {
        content: "Note: ";
        color: red; font-weight: bold; }
</style>
<p>Submit your assignment on time.</p>
```

Output: Note: Submit your assignment on time.

The text Note: is added using CSS. It is not written inside the HTML paragraph.

4. ::after

```
<style>
    p::after {
        content: " ✓";   color: green; }
</style>
<p>Task completed</p>
```

Output: Task completed ✓

The check mark appears after the paragraph's text.

5. ::placeholder

```
<style>
    input::placeholder {
        color: gray;
        font-style: italic;}
</style>
<input type="text" placeholder="Enter your email">
```

Output: The placeholder text “Enter your email” appears gray and italic.

6. ::selection

```
<style>
    p::selection {
        background-color: yellow;
        color: black;  }
</style>
<p>Select this text using your mouse.</p>
```

Output: When the user selects the paragraph text, the selected portion gets a yellow background with black text.

Creating Responsive and Interactive Web Pages

CHITKARA UNIVERSITY

➢ Introduction

•A responsive web page adjusts according to the screen size.
•An interactive web page responds to user actions.
•HTML creates the structure of a web page.
•CSS is used for styling and responsive design.
•JavaScript adds interactive features.

➢ Responsive Web Design

Responsive web design makes a website look good on mobile phones, tablets, and computers.

Advantages:

- Better user experience.
- Easy to use on different devices.
- Less horizontal scrolling.
- Improved readability.

➢ Techniques for Responsive Design

•**Flexible layouts:** Adjust according to screen size.
•**Flexible images:** Resize images to fit the screen.
•**Media queries:** Apply different CSS styles for different screen sizes.

Example:

```
img {
    max-width: 100%;
    height: auto;
}
```

➢ CSS Media Queries

Media queries help change the appearance of a web page according to screen width.

```
body {  background-color: lightblue;  }
@media (max-width: 600px) {
    body {
        background-color: lightyellow;
    }}
```

Interactive Web Pages

An interactive web page responds to user actions.
Examples:
•Clicking a button.
•Hovering over a link.
•Filling out a form.
•Selecting a checkbox.
•Opening a menu.

➢ **CSS Hover Effect**

The :hover pseudo-class applies a style when the mouse pointer moves over an element.

```
<button>Click Me</button>
button:hover { background-color: green; color: white;}
```

Output: The button changes to green when the mouse pointer moves over it.

➢ **CSS Transitions**

Transitions make changes in CSS properties appear smooth.

```
button {
    background-color: blue; transition: 0.5s; }
button:hover { background-color: red; }
```

Output: The button smoothly changes from blue to red when hovered over.

MCQ Based Questions

Q. Responsive Web Design

What will happen when the screen width becomes **600px or less**?

```
.container
{
display: flex;
 }
@media (max-width: 600px)
{
.container
{
flex-direction: column;
}
}
```

A. The container disappears

B. Items change from row to column

C. Items become hidden

D. Font size automatically increases

Answer: B

MCQ Based Questions

Q. CSS Animation

What will this animation do?

```
.box {
 animation: move 2s infinite;
}
@keyframes move {
 from {
transform: translateX(0);
}
to {
transform: translateX(100px); }
}
```

A. The box moves 100px to the right repeatedly

B. The box moves 100px to the left once

C. The box rotates continuously

D. The box changes its width

Answer: A